

Computational Thinking



Aniel Bhulai
Vrije Universiteit Amsterdam

Contents

Preface	6
1. What is computational thinking.....	7
2. Solution strategies	8
2.1. Easy vs. efficient solution strategy.....	8
2.2. How to approach a problem?	8
3. Examples of solution strategies	10
3.1. The “Guess and Check” solution strategy.....	10
3.2. The “Go through all the possibilities” solution strategy	10
3.3. The “Divide the problem into several sub-problems” solution strategy	10
3.4. The “Use of formulas or equations” solution strategy.....	10
3.5. The “Discover a structure or pattern” solution strategy.....	10
3.6. The “Make a model” solution strategy	11
3.7. The “Brute force” solution strategy.....	11
3.8. The “Divide-and-conquer” solution strategy	11
4. Algorithms	13
4.1. The importance of knowing something about algorithms.....	13
4.2. What is an algorithm?.....	13
4.3. Properties of algorithms	15
4.4. Pseudocode	15
5. Big O	17
5.1. Big O notation.....	17
5.2. Understanding Big O	18
5.3. Determining complexities.....	20
5.3.1. Sequence of statements.....	20
5.3.2. if-else.....	20
5.3.3. Loops	21
5.3.4. Nested loops.....	21
5.3.5. Statements with function or procedure calls	21
6. Search algorithms	23
6.1. Introduction.....	23
6.2. Linear search	23

6.2.1.	Analysis.....	23
6.3.	Binary search	24
6.3.1.	Analysis.....	25
7.	Sorting algorithms.....	26
7.1.	Introduction.....	26
7.2.	Classification.....	27
7.3.	Bubble sort	28
7.3.1.	Analysis.....	28
7.4.	Merge sort	29
7.4.1.	Analysis.....	30
7.5.	Quicksort	30
7.5.1.	Partitioning.....	31
7.5.2.	Analysis.....	31
8.	Greedy technique	33
9.	Introduction to graph theory	35
9.1.	Introduction.....	35
9.2.	Graphs	35
9.3.	Representations of graphs.....	36
9.4.	Some examples of graphs.....	37
9.4.1.	Weighted graphs	37
9.4.2.	Simple graphs	38
9.4.3.	Multigraph.....	38
9.4.4.	Connected and disconnected graphs.....	38
9.4.5.	Connected component.....	39
9.4.6.	Complete graphs	39
9.4.7.	Cycle graphs	40
9.4.8.	Trees.....	40
9.4.9.	Eulerian graph	41
9.4.10.	Hamiltonian graph	42
10.	Graph algorithms	43
10.1.	Dijkstra's algorithm	43
10.1.1.	How does Dijkstra's algorithm work?	43

10.2.	Prim's algorithm.....	45
10.3.	Kruskal's algorithm.....	46
10.3.1.	Comparison of Prim's and Kruskal's algorithm	46
References		48
Index		51

Preface

The Computational Thinking course offers an introduction to problem-solving both by reasoning and using knowledge from other disciplines. In this course, you will be first introduced to different general solution strategies such as guess and check, creating models, and formulation. A specific way of solving problems is by using algorithms, for instance, search algorithms, sorting algorithms, and graph algorithms. The lectures and classes will offer a theoretical overview of the different strategies and algorithms. Meanwhile, in the practical sessions, you will get the chance to put that knowledge into practice to solve diverse problems. There are many ways to solve a problem. This course aims to encourage you and provide you with the necessary tools to start developing algorithms by yourself.

The Computational Thinking course aims to teach you:

- ✓ how to analyze problems,
- ✓ how to choose the right solution strategy or algorithm to solve problems,
- ✓ how to create algorithms, and
- ✓ how to translate algorithms into a flowchart.

For this course, we expect that you have some basic knowledge of the Python programming language.

1. What is computational thinking

When facing a problem, we can use computers to reach a solution. However, before we can tackle a problem, we need to understand it and the different ways to solve it. Here is where Computational Thinking (CT) comes into play. We can think of computational thinking as a set of problem-solving methods. In computational thinking, problems and their solutions are expressed in such a way that a human and a computer can effectively execute (Wing, 2014).

As such, Computational Thinking is not only about solving problems but also about formulating them. It allows us to take a complex problem, understand it (e.g., by decomposition), and develop possible solutions (e.g., by pattern recognition). Computational thinking facilitates the abstraction process. That is, focusing on the important information only and ignoring the irrelevant details. By doing this, we can develop a step-by-step solution to the problem (i.e., an algorithm) in a way that both a human and a computer can understand and execute. Finally, these steps are used to program (e.g., with Python) a computer to solve the complex problem.

2. Solution strategies

2.1. Easy vs. efficient solution strategy

There are several strategies to solve everyday problems. Often, there are different ways of solving a problem. There is not always a “best way”. However, sometimes one way might be more efficient, or you might find one approach easier or more pleasant than the other.

Consider the following problem of a tennis club facing financial issues (see Problem 2-1).

Problem 2-1

A tennis club is in financial problems. Its members want to help the club by donating money. In total, there are one hundred members prepared to donate money. Each of the one hundred people donates a certain amount of money in the following way. Person 1 donates 1 Euro, person 2 donates 2 Euros, person 3 donates 3 Euros, person 4 donates 4 Euros, etc.

Calculate (without a calculator) how much money the members donate together.

Problem 2-1: Members donate money to the tennis club.

To solve this problem without using a calculator we can add up all the donated Euros of the 100 members. This means we would have to make the following calculation:

The total amount of donated money = $1 + 2 + 3 + \dots + 98 + 99 + 100$. eq. (2-1)

This solution strategy is simple, but also time-consuming. Besides that, it is not efficient. And what if we had a thousand members or a million members instead of a hundred members. Would you still use the same solution strategy?

As you have probably guessed, we would need a different solution strategy to solve this problem more efficiently. As mentioned before, there are several ways of solving problems. To find a more efficient solution strategy, we need to know how to approach problems. Therefore, in the next section, we will discuss a scheme that will help us approach problems systematically.

2.2. How to approach a problem?

To solve a problem efficiently it is necessary to approach the problem systematically. Systematic means that we have to follow steps to solve the problem. Figure 2-1 shows the five steps required to solve a problem.

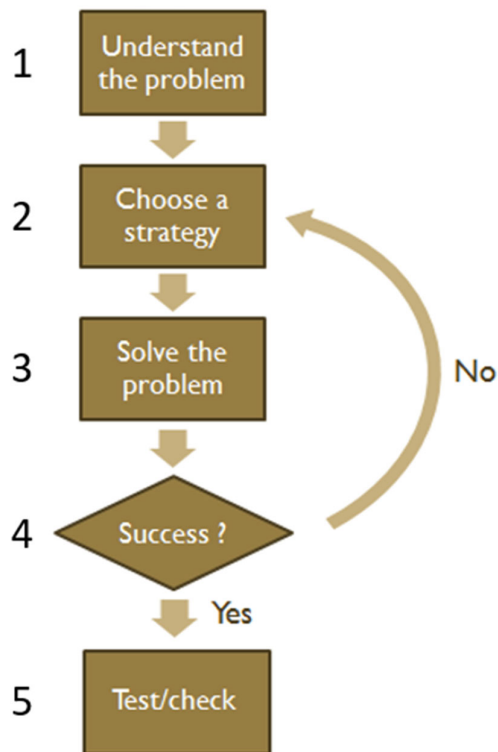


Figure 2-1: Five steps to approach a problem

The *first step* in approaching the problem is to have a good understanding of the problem. That is, can we formulate the problem in our own words?, Do we understand the problem properly?, Do we know what we have to solve?, Have we seen such a problem before?, Which information does the problem provide us, and what information do we need to solve the problem?

In the *second step*, we choose the right solution strategy (or a combination of a few strategies) to solve the problem. In the *third step*, we solve the problem using the chosen strategy.

The *fourth step* is to check whether the solution strategy worked or not to solve the problem. If the solution strategy did not work, we must go back to the second step and choose a different solution strategy.

Even if we think that we have solved the problem, we should check our solution in the *fifth and last step*. For instance, we could have made miscalculations or fallacies. Sometimes we cannot test our solution. In those cases, we have to check whether the solution we found is reasonable or not (for example, check the size of the number).

In the next chapter, we will discuss some solution strategies.

3. Examples of solution strategies

There are many solution strategies for solving problems. Here, we will discuss the following solution strategies.

- 1) The “Guess and Check” solution strategy
- 2) The “Go through all the possibilities” solution strategy
- 3) The “Divide the problem into several subproblems” solution strategy
- 4) The “Use of formulas or equations” solution strategy
- 5) The “Discover a structure or pattern” solution strategy
- 6) The “Make a model” solution strategy
- 7) The “Brute force” solution strategy
- 8) The “Divide-and-conquer” solution strategy

3.1. The “Guess and Check” solution strategy

The “Guess and Check” solution strategy is also known as the “Try something” solution strategy.

We can use it for problems that are not too complex and that we have an overview of.

3.2. The “Go through all the possibilities” solution strategy

The go through all the possibilities solution strategy can be used when the number of possibilities to solve a problem is limited. For example, if there are only four possibilities to solve a problem, then we can simply try all four possibilities.

3.3. The “Divide the problem into several sub-problems” solution strategy

Sometimes a problem looks very complex. But when you divide the problem into sub-problems, the problem becomes easier to solve. This is precisely what the “divide the problem into several sub-problems” solution strategy is. There are several approaches to divide a problem into sub-problems such as simplifying, back reasoning, or exclusion. We could also combine some of these approaches to reduce the complexity of the problem, making it simpler and hence easier to solve.

3.4. The “Use of formulas or equations” solution strategy

Sometimes it is easy to solve a problem by using formulas and equations. In this way, we can have a better overview of the problem. Thus complex problems become easier to handle and solve.

3.5. The “Discover a structure or pattern” solution strategy

Some problems can be solved by discovering a structure or a pattern in similar problems. Thus another way of solving problems is to look at other similar problems to discover a structure or pattern. Once the structure or pattern is known, it becomes easier to solve the problem.

3.6. The “Make a model” solution strategy

Mathematical models are often used to solve complex problems. When we model a problem, we omit many of the aspects, that makes the problem complex. Thus, we simplify the problem and can then solve it with known and simple techniques.

Examples of modeling are making a diagram or drawing a picture. Such models can help us discover things that lead to a solution to the problem.

3.7. The “Brute force” solution strategy

The brute force solution strategy is a simple approach to solve problems. It relies on sheer computing power to try out all possibilities until a solution is found. Therefore, brute force does not use any algorithm or heuristics¹ to speed up the calculation of the solution. Brute force is often used if no algorithm is known to reach a solution faster or more efficiently. The linear search algorithm and the bubble sort algorithm are examples of the brute force solution strategy. We will discuss these algorithms in detail in Chapter 6 and Chapter 7.

3.8. The “Divide-and-conquer” solution strategy

The divide-and-conquer solution strategy (D&C) is an important and probably the best-known general technique to design algorithms. D&C consists of three major steps (see also Figure 3-1). In the first step, we divide a problem of n size into a number of small sub-problems of the same type and ideally of about the same size.

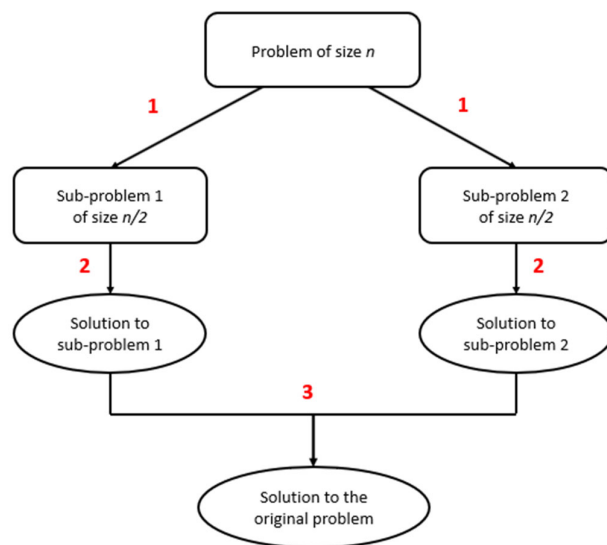


Figure 3-1: Divide-and-Conquer technique

¹ A heuristic is a rule or method that helps us solve problems faster than we would if we did all the computing (Heuristic, n.d.). We can think of heuristic as a mental shortcut that allows people to solve problems and make decisions quickly and efficiently. These rule-of-thumb strategies shorten decision-making time and are helpful in many situations, but they can also lead to systematic errors in thinking (Cherry, n.d.). Thus, heuristics can speed up our problem and decision-making process, but they can introduce errors. There is no guarantee that something that worked in the past will work again. In addition, it can be difficult to see alternative solutions or come up with new ideas when relying on an existing heuristic.

Hence, to apply D&C, we divide the original problem into sub-problems until these become simple enough to be solved directly. In the example, in Figure 3-1, the problem of size n is divided into sub-problem 1 and sub-problem 2, each of size $n/2$. In the next step of the D&C technique, we solve each sub-problem recursively. Finally, in the third step, we combine all these solutions to get a solution to the original problem.

A typical divide-and-conquer algorithm solves a problem using the following three steps:

1. Divide: Divide the given problem into sub-problems of the same type.
2. Conquer: Solve these sub-problems recursively.
3. Combine: Combine the solutions appropriately.

It is good to know that the D&C technique is ideally suited for parallel computations. In parallel computations, each sub-problem can be solved simultaneously by its own processor.

In Chapter 6 and Chapter 7 we will discuss the binary search, the merge sort, and the quick sort algorithms that apply the D&C technique.

4. Algorithms

4.1. The importance of knowing something about algorithms

Why is it so important to know something about algorithms? There are both practical and theoretical reasons to study algorithms if you are considering working later as a computer professional. From a practical point of view, you should know a standard set of important computer science algorithms. Besides knowing the different algorithms, you should also be able to design new algorithms and analyze their efficiency. From a theoretical point of view, the study of algorithms is recognized as the pillar of computer science.

However, even if you are not a student in a computing-related program, there are several reasons why having some knowledge of algorithms might benefit you. One reason is that computer applications are integrated into many things today, such as mobile phones, tablets, watches, and wearables. Without algorithms, no computer program would exist, and without computer programs, we would not be able to make those beautiful devices. Computer programs have become indispensable in many aspects of our personal and professional lives. Studying algorithms is therefore becoming a necessity for more and more people.

Another reason to study algorithms is that we develop analytical skills, which are necessary to solve problems. Algorithms can be considered as special kinds of solutions to problems – not just answers but precisely defined procedures for getting answers (Levitin, 2012, p. 27). Therefore, specific algorithm design techniques can be interpreted as problem solving strategies. Donald Knuth, one of the most prominent computer scientists in the history of algorithms, put it this way:

“A person well-trained in computer science knows how to deal with algorithms: how to construct them, manipulate them, understand them, analyze them. This knowledge is preparation for much more than writing good computer programs; it is a general-purpose mental tool that will be a definite aid to the understanding of other subjects, whether they be chemistry, linguistics, or music, etc. The reason for this may be understood in the following way: It has often been said that a person does not really understand something until after teaching it to someone else. Actually, a person does not really understand something until after teaching it to a computer, i.e., expressing it as an algorithm... An attempt to formalize things as algorithms leads to a much deeper understanding than if we simply try to comprehend things in the traditional way.” (Knuth D. E., 1996, p. 9).

4.2. What is an algorithm?

What is an algorithm? Formally we can say that

An algorithm is an effective method of solving a problem or performing a task that consists of a finite number of steps or instructions.

Algorithms need not be overly complicated. Actually, you might already be using algorithms to perform tasks in your everyday life without noticing it. For instance, you may have an algorithm for going from home to school:



Figure 4-1: Going from home to school.

- Step 1: Walk to the bus station**
- Step 2: Take the bus**
- Step 3: Walk to school**

or for efficiently going through your shopping list in a grocery store:



Figure 4-2: Shopping list.

- Step 1: Scan the list from top to bottom**
- Step 2: Find the item that you want to buy**
- Step 3: Cross out the item that is bought**

or for preparing for your exam:



Figure 4-3: Preparing your exam.

- Step 1: Attend lectures**
- Step 2: Study slides**
- Step 3: Make assignments**
- Step 4: Study textbooks**
- Step 5: Make summaries**

An important thing about an algorithm is that it always works if we follow the steps. It's like a recipe for baking a cake. If you follow the recipe correctly, you will always get a delicious cake. In a nutshell, an algorithm gives us a roadmap to solve a problem or to accomplish a task. Algorithms are solution strategies that *always* lead to a solution. A solution strategy,

on the other hand, does not necessarily always work. So we can say that all algorithms are solution strategies, but not all solution strategies are algorithms. Note that Figure 2-1, the steps to approach a problem, is in fact an algorithm, namely an algorithm to approach and solve problems.

4.3. Properties of algorithms

An algorithm is useful when it helps us find a solution to a specific problem. For that to happen, an algorithm must meet the following properties.

- a) **Input:** Typically, an algorithm takes input values from a specified set of elements, specifying the amount and type of input.
- b) **Output:** The algorithm produces the output values for the specified set from the input values. These output values are the solution for the specific problem.
- c) **Definiteness:** The steps of the algorithm must be precisely defined. This means that there is no ambiguity.
- d) **Correctness:** The algorithm must produce the correct output values for each set of input values.
- e) **Finiteness:** The output values must be produced after a finite number of steps for any input values in the specified set.
- f) **Effectiveness:** It must be possible to perform each step of the algorithm correctly and within a reasonable time.
- g) **Generality:** The algorithm should work for all problems of the desired form, not just for a particular set of input values.

When an algorithm meets these properties, it is a fail-proof way to solve the problem for which it was designed.

4.4. Pseudocode

Suppose we want to design an algorithm for counting the number of people in a classroom. You would probably point at each person one at a time and start counting from 0, 1, 2, 3, 4, 5 and so on. Well, that's an algorithm. When designing an algorithm, it is a good habit to express the algorithm a bit more formally in what we call *pseudocode*. Pseudocode is an English-like syntax similar to a programming language. It is a detailed yet readable description of what a computer program or algorithm should do. The pseudocode is expressed in a formally formatted natural language rather than in a programming language. An example of pseudocode for counting people in a classroom could be:

```
1| Let NumberOfPerson = 0
2| for each person in the classroom
3|   set NumberOfPerson = NumberOfPerson + 1
```

How do we interpret this pseudocode? In line 1, we declare, as it were, a variable named *NumberOfPerson*, and we initialize its value to zero. This just means that at the beginning of our algorithm, the variable *NumberOfPerson* (the thing we're counting with) has a value of zero. After all, before we start counting, we haven't counted anything yet. Calling this variable *NumberOfPerson* is just a convention. We could have called it almost anything. In line 2, we mark the start of a *loop*. A loop is a sequence of steps that is repeated a number of times. In our example, the step we take is to count people in the classroom. Line 3 describes exactly how we count. The indentation means that line 3 is repeated. So, what the pseudocode says is that after we start from zero, for each person in the classroom, we will increase the variable *NumberOfPerson* by 1.

Writing pseudocodes has some advantages. One of the advantages is that catching errors (i.e., debugging) at the pseudocode stage is less costly than debugging them later in the development process. Another advantage is that the pseudocode can be inspected by a team of designers and programmers to ensure that actual programming is likely to match the design specifications.

5. Big O

5.1. Big O notation

In computer science, the *big O notation* (with a capital *O*, not a zero), also called *Landau's symbol*, is used to classify algorithms based on how they respond (e.g., in their processing time or workspace requirements) to changes in input size (Mohr, 2010). It characterizes functions based on their growth rates: different functions with the same growth rate can be represented with the same big *O* notation (e.g., $f(x) = x^2 + 2x + 8$ and $g(x) = 5x^2 + 10x + 100$). The big *O* notation describes the asymptotic behavior of functions as x grows to infinity. *Landau's symbol* comes from the name of the German number theoretician Edmund Landau who invented the notation (Landau, 1909). The letter *O* is used because the growth rate of a function is also called the *order of the function*. A description of a function in terms of big *O* notation usually only gives an upper bound on the growth rate of the function (Big O notation, n.d.).

Big *O* notation is useful when analyzing algorithms for efficiency. Suppose we have an algorithm that completes a problem of size n in the time (or a number of steps) $T(n)$, where $T(n)$ is defined as $T(n) = 5n^2 - 2n + 10$. Then the n^2 term will dominate when n grows large so that we can ignore all the other terms. Let for instance $n = 400$, then the term $5n^2$ will be 1000 times as large as the $2n$ term (800000 vs 800). Ignoring the $2n$ term would have a negligible effect on the expression's value for most purposes. Also, when we compare to any other order of expressions, for example, an expression containing a term n^3 or n^4 , then the coefficients (in our example 5, 2, and 10) become irrelevant. So, for our example, we could say that " $T(n)$ grows at the order of n^2 " and we write: $T(n) = O(n^2)$. Note that the "=" sign is not meant to express "is equal to" in its normal mathematical sense, but the "=" sign is more an informal "is". We can also write $T(n) = O(n^2)$ as $T(n) \in O(n^2)$ which is sometimes considered more accurate (Cormen, Leiserson, & Rivest, 2009).

Table 5-1 shows some classes of functions that often occur when analyzing the running time of an algorithm. The slower-growing functions are listed first. In each case, c is an arbitrary constant, and n increases without bound (i.e., $n \rightarrow \infty$).

Running time ($T(n)$)	Name	Examples of running times	Example algorithms
$O(1)$	Constant time	10	Determining if an integer (represented in binary) is even or odd.
$O(\log(n))$	Logarithmic time	$\log(n^2), \log(n)$	Binary search
$O((\log(n))^c), c > 1$	Polylogarithmic time	$(\log(n))^2$	Matrix chain ordering ² can be solved in polylogarithmic time on a Parallel Random Access Machine.
$O(n)$	Linear time	n	Finding the smallest item in an unsorted array.
$O(n^2)$	Quadratic time	n^2	Bubble sort; Insertion sort
$O(n^c), c > 1$	Polynomial time	$n, n \log n, n^{10}$	Tree-adjointing grammar parsing; maximum matching for bipartite graphs
$O(c^n), c > 1$	Exponential time	$2^n, 2^{n^2}$	Finding the (exact) solution to the traveling salesman problem using dynamic programming; determining if two logical statements are equivalent using brute-force search.

Table 5-1: Big O notations (Time complexity, n.d.) (Landau, 1909)

Note that $O(n^c)$ and $O(c^n)$ are very different. The latter grows much, much faster, regardless of how big the constant c is. A function that grows faster than any power of n is called *superpolynomial*. A function that grows slower than an exponential function of the form c^n is called *sub-exponential*. An algorithm can require time that is both superpolynomial and sub-exponential. Examples of this include the fastest algorithms known for integer factorization.

Also, note that $O(\log(n))$ is exactly the same as $O((\log(n))^c)$. The logarithms differ only by a constant factor that is ignored by the big O notation. Similarly, logs with different constant bases are equivalent.

5.2. Understanding Big O

In the previous section, we saw that the Big O notation is useful when analyzing algorithms for efficiency. Many resources are involved in efficiency, including:

- CPU (time) usage
- Memory usage
- Disk usage
- Network usage

All resources are important, but we will mainly talk about *time complexity* (CPU usage) expressed by the big O notation when we discuss the efficiency of an algorithm or a piece of code. Note that there is a difference between the performance and time complexity of an algorithm. The performance is about how much time, memory, disk, etc., is actually used

² The matrix chain ordering - also called matrix chain multiplication - problem is an optimization problem to multiply a given sequence of matrices. The problem is not actually to perform the multiplications, but merely to decide in which order to perform the multiplications. Matrix chain ordering problem can be solved using dynamic programming (Matrix Chain Multiplication | DP-8, 2021), (Matrix chain multiplication, n.d.).

when a program is running. This depends on the machine, compiler, etc. as well as the code. Time complexity, on the other hand, refers to how the resource requirements of a program or algorithm scale, i.e., what happens when the size of the problem to be solved increases (Lundqvist, n.d.)? Note that time complexity affects performance but not the other way around.

The time required by a function or procedure is proportional to the number of “basic operations” that it performs. Some examples of basic operations are:

- One arithmetic operation (e.g., +, *, /).
- One assignment (e.g., $x = 5$)
- One test (e.g., $x == 5$)
- One read (of a primitive type: integer, float, character, Boolean)
- One write (of a primitive type: integer, float, character, Boolean)

Some functions or procedures perform the same number of operations every time they are called. We say that these functions or procedures take *constant time*. Other functions or procedures may perform a different number of operations depending on the value of a parameter. For example, in the bubble sort algorithm, the number of elements in the list determines the number of operations performed by the algorithm. This parameter (i.e., the number of elements) is called the *problem size* or *input size* (Lundqvist, n.d.).

We are not interested in the exact number of operations that are being performed when we try to find the complexity of an algorithm (e.g., a program, function, or procedure). Instead, we are interested in the relationship between the number of operations and the problem size.

Most of the time, we are interested in the worst-case time complexity, denoted as $T(n)$. In other words, we are interested in the maximum number of operations that can be performed for a given problem size. For example, when we insert an element into an array, we have to move the current element and all elements that come after it one place to the right in the array. In the worst-case, inserting at the beginning of the array, all elements in the array must be moved. Therefore, at worst, the insertion time is proportional to the number of elements in the array, and we say that the worst-case time complexity for the insertion operation is linear in the number of elements in the array. For a linear-time algorithm, if the problem size doubles, the number of operations also doubles (Lundqvist, n.d.).

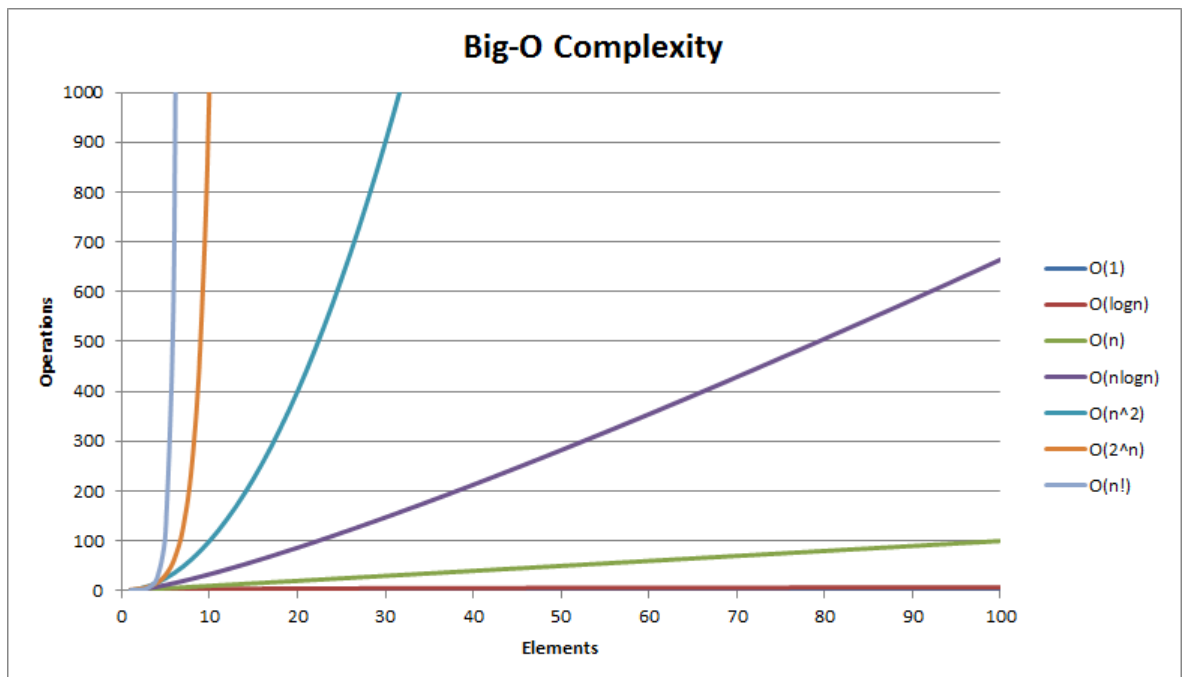


Figure 5-1: Big-O complexity (Rowell, n.d.)

5.3. Determining complexities

In general, to determine the running time of a piece of code, we have to look at the statements, since the kind of statements used in the code determines the running time. Below we give some examples of statements and running time.

5.3.1. Sequence of statements

Suppose we have a sequence of statements such as the following:

```
statement 1
statement 2
...
statement k
```

Then the total time is found by adding the times for all the statements

$$\text{total time} = \text{time}(\text{statement 1}) + \text{time}(\text{statement 2}) + \dots + \text{time}(\text{statement } k).$$

If each statement is “simple” (i.e., it only involves basic operations) then the time for each statement is constant and the total time is also constant, which means that $T(n) = O(1)$.

5.3.2. if-else

Suppose that we have the following if-condition

```
if (condition) then
    block 1 (sequence of statements)
else
    block 2 (sequence of statements)
end if
```

In this example, either block 1 will execute, or block 2 will execute. Therefore, the worst-case time complexity is the slower of the two possibilities

$\max(\text{time}(\text{block 1}), \text{time}(\text{block 2})).$

If block 1 takes $O(1)$ and block 2 takes $O(N)$, then the time complexity of the if-else statement would be $O(N)$.

5.3.3. Loops

Suppose we have the following loop

```
for  $i$  in  $1..n$  loop
    sequence of statements
end loop
```

In the example, the loop executes n times, so the sequence of statements also executes n times. If we assume that all the statements are $O(1)$, then the total time for the for-loop is $n \times O(1)$, which is $O(n)$ overall.

5.3.4. Nested loops

Suppose we have the following nested loop

```
for  $i$  in  $1..n$  loop
    for  $j$  in  $1..m$  loop
        sequence of statements
    end loop
end loop
```

In the example, the outer loop executes n times. Every time the outer loop executes, the inner loop executes m times. As a result, the statements in the inner loop execute a total of $n \times m$ times. Thus, the time complexity is $O(n \times m)$.

5.3.5. Statements with function or procedure calls

When a statement includes a function or procedure call, the time complexity of the statement includes the time complexity of the function or procedure. Suppose that we know that a function or procedure f takes constant time and that a function or procedure

g takes time proportional to (linear in) the value of its parameter k . Then, the statements below have the following time complexities:

$f(k)$ has $O(1)$
 $g(k)$ has $O(k)$.

Then when a loop is involved, the same rule applies, e.g.,

```
for  $j$  in  $1..n$  loop  
     $g(j)$   
end loop
```

The above example has a time complexity of $O(n^2)$. The loop executes n times and each function or procedure call $g(n)$ has a time complexity of $O(n)$.

6. Search algorithms

6.1. Introduction

Searching for a keyword or value is the basis of many computing applications, whether it is in an internet search engine or a contact list on your mobile phone. In those situations, we consciously or unconsciously use a search algorithm or some strategy to find values. In computer science, a search algorithm is an algorithm for finding an item with specified properties among a collection of items (Search algorithm, n.d.). The items may be stored individually as records in a database or maybe as elements of a search space³. There are many search algorithms. And there is also no single algorithm for searching that fits all situations best. Some algorithms are faster than others but require more memory. Some algorithms are fast, but can only be applied to sorted arrays, and so on. Unlike sorting algorithms, searching algorithms have no stability problems (Levitin, 2012).

6.2. Linear search

Linear search - also known as *sequential search* - is the simplest and most basic search algorithm for finding a particular element or value in a list. Linear search belongs to the Brute force strategy and is only efficient for small lists. For large lists ($n > 8$) binary search is more efficient. Linear search sequentially moves through a list and checks each element until the desired element (the so-called *key*) is found or the list is exhausted. When linear search finds the desired element, it returns the location of the element and stops searching further. For linear search, the list does not need to be sorted. Figure 6-1 shows the linear search pseudocode.

```
FOR each element in the list
    check if element is equal to the element being searched
        IF element found
            stop with searching and return the element's location
        ELSE continue
stop
```

Figure 6-1: Linear search pseudocode

6.2.1. Analysis

If we look at the performance of linear search, we see that for the best case the algorithm needs only one comparison to search the key element in a list with n elements. The best case occurs when the key element is in the first position of the list. In the worst-case, the key element is either in the last position or not present in the list. This means that in the worst-case the algorithm needs n comparisons if there are n elements in the list.

³ A search space is a feasible region defining the set of all possible solutions.

If the searched value occurs k times in the list, and all order of the list is equally likely, then the expected number of comparisons is

$$\begin{cases} n, & \text{if } k = 0 \\ \frac{n+1}{k+1}, & \text{if } 1 \leq k \leq n \end{cases} \quad \text{eq. (6-1)}$$

The worst-case cost and the expected cost of the linear search are both $O(n)$.

6.3. Binary search

The *binary search algorithm* (also called the *half-interval search algorithm*) belongs to the Divide-and-Conquer solution strategy, which we discussed in Chapter 3. Binary search is an efficient algorithm for searching the position of an element in a sorted list. A sorted list means that the elements in the list are arranged either in ascending order or in descending order. The average speed of binary search is faster than the linear search.

The binary search algorithm is deceptively simple. Pretend someone is thinking of a number between 1 and 100. With every guess we take, the person says higher or lower. The most efficient way to discover the person's number is to guess 50 first. After the person says "higher", we would guess 75. Then, the person says "lower" and we guess 63. Finally, the person says "higher" and we guess 69. The person says "Yes!".

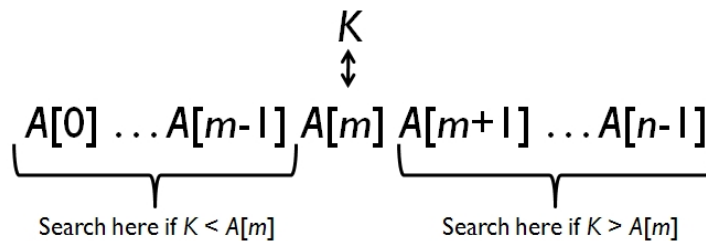


Figure 6-2: Searched value by binary search.

Consider the list in Figure 6-2. The binary search algorithm starts by comparing a search key K with the middle element $A[m]$ of the list. If they match, the algorithm stops. If the search key K is greater than the middle element $A[m]$, then the first half of the list is eliminated and the algorithm continues with the same operation recursively for the second half of the list. If the search key K is less than the middle element, then the second half of the list is eliminated and the algorithm continues with the same operation recursively for the first half of the list. In this way, the binary search algorithm reduces the number of elements needed to be checked by a factor of two each time and finds the searched value if it exists in the list or if not, it determines "not present", in logarithmic time.

Thanks to its performance characteristics over large collections, the binary search algorithm is often used. The only time that binary search makes no sense is when the collection is frequently updated (relative to searches) since re-sorting is required.

6.3.1. Analysis

The standard way to analyze the efficiency of binary search is to count the number of times the search key K is compared with an element of the list. Moreover, for simplicity, we will count the so-called three-way comparisons (Levitin, 2012). This assumes that after one comparison of K with $A[m]$, the algorithm can determine whether K is less than, equal to, or greater than $A[m]$.

When we count the three-way comparisons, we see that in the best case we just need 1 comparison to find the search key K . In the best case, the search key K is equal to the middle element $A[m]$. In the worst-case, we need $\lceil \log_2(n + 1) \rceil$ comparisons and in the average case we need about $\log_2(n)$ comparisons. Note that in the worst-case we take the ceiling⁴ of $\log_2(n + 1)$.

The runtime complexity of the binary search algorithm is $O(1)$ in the best-case. For both in the worst-case and the average case, the runtime complexity of binary search is $O(\log(n))$.

⁴ The floor and ceiling functions are defined as $\text{floor}(x) = \lfloor x \rfloor$ is the largest integer not greater than x and $\text{ceiling}(x) = \lceil x \rceil$ is the smallest integer not less than x (Graham, Knuth, & Patashnik, 1994).

7. Sorting algorithms

7.1. Introduction

Sorting as a concept is deeply rooted in many of the things we do. It often happens that we want to arrange things or data in a given order. We do this to improve the readability of data or to facilitate the search and extraction of information from that data.

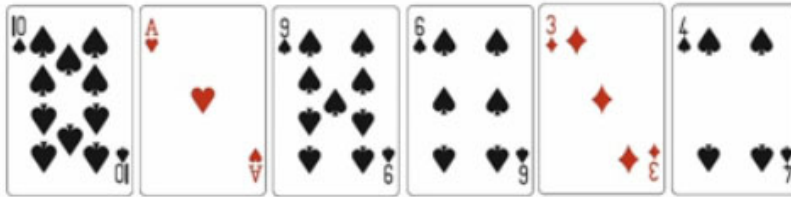


Figure 7-1: Unsorted hand of cards

Take the simple example of playing a card game. When we play a card game, we like to keep our hands of cards sorted by rank or suit, even though the number of cards in our hands is relatively small. So, if our hand of cards is as shown in Figure 7-1 and we would like to keep it in ascending order of rank, the ranking will look something like Figure 7-2.

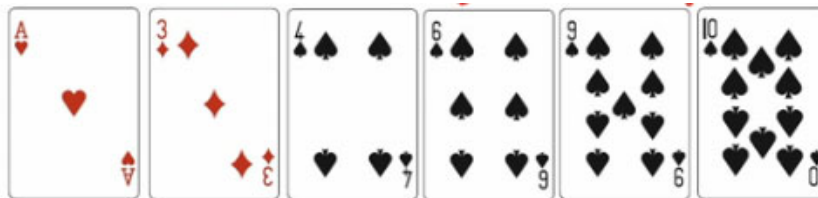


Figure 7-2: A sorted hand of cards

Sorting is a handy feature. For instance, we might sort the words in a dictionary to find them easily, or sort a salary table to see who has the highest salary.

A formal definition of sorting could be the following:

Sorting is the arrangement of the elements in a list or collection in ascending or descending order of a property. The list must be homogeneous, i.e., all its elements must be of the same type.

Sorting efficiently is important to optimize the use of other algorithms that require sorted lists to work correctly. Efficient sorting is also crucial to normalize and standardize data that can be presented in more than one way. This is often the case when we work with databases. Sorting makes searching easier. In addition, sorting produces human-readable output.

When we study sorting algorithms, we usually use a list of integers, and generally, we sort the list of integers in ascending order of value. Suppose that we have a list of integers, as shown in Figure 7-3.

3	4	10	5	7
----------	----------	-----------	----------	----------

Figure 7-3: Integers in unsorted order

Then, if you sort it in ascending order of value, you need to rearrange the elements as shown in

Figure 7-4.

3	4	5	7	10
----------	----------	----------	----------	-----------

Figure 7-4: Integers in an increasing order of value

Sorting the list in descending order of value would result in an order as in Figure 7-5.

10	7	5	4	3
-----------	----------	----------	----------	----------

Figure 7-5: Integers in decreasing order of value

And as we said in the definition, we can sort the list by any property. What if we want to sort this according to, let's say, an increasing number of factors? In this case, the integer with fewer factors would be towards the beginning of the list.

3	5	7	4	10
----------	----------	----------	----------	-----------

Figure 7-6: Integers in an increasing order of number factors

As we can see from Figure 7-6, 3 has only 2 factors, 1 and 3 itself. 4 has 3 factors, 1, 2, and 4 itself. 5 also has 2 factors, 1 and 5 itself. 7 has two 2 factors, 1 and 7 itself, 10 has 4 factors, 1, 2, 5, and 10 itself.

As we can see, a sorted list is a permutation of the original list. When we sort a list, we only rearrange the elements. Sorted data is suitable not only for presentation or manual retrieval of information, but it is also advantageous when we use the computing power of machines.

7.2. Classification

Sorting algorithms used in computer science can be classified according to several criteria. A first possibility is to categorize them based on how the list size affects the computational complexity of element comparisons. Computational complexity, in this case, refers to the worst, average, and the best behavior of element comparisons. Other possible classifications are according to memory usage, use of other computer resources, and recursion. Some algorithms are either recursive or non-recursive, while others may be both (e.g., merge sort).

A further option is to classify depending on the stability. Stable sorting algorithms maintain the relative order of records with equal keys (i.e., the values). We say that a sorting algorithm is *stable* if the relative order of the elements with equal values in the input is maintained in the output. When all values are different, this distinction is not necessary. However, if there are equal values, a sorting algorithm is stable if whenever there are two

records (let's say R and S) with the same key and R appears before S in the original list, then R will always appear before S in the sorted list.

Lastly, a sorting algorithm is said to be *in place* if the algorithm does not require extra memory to store temporary data. These algorithms do not need to create auxiliary locations to store data temporarily, as is the case with other sorting algorithms.

7.3. Bubble sort

Bubble sort, also known as *sinking* sort, belongs to the Brute force strategies. Bubble sort is a relatively straightforward data sorting method. It is an in-place and stable algorithm. The bubble sort gets its name because elements tend to move up in the correct order, like bubbles that rise to the surface. Bubble sort compares the first two adjacent elements of an array and swaps them if they are out of order. It continues doing this for each adjacent element until it ends at the last entry. Formally, we can say that the bubble sort algorithm starts at the beginning of a data set. Then the first two adjacent elements are compared, and if the first element is greater than the second, they are swapped. The algorithm continues to do this for each pair of adjacent elements up to the end of the data set. It then starts over with the first two elements and repeats until no swaps have occurred on the last pass.

7.3.1. Analysis

When we consider the performance of bubble sort for a list with n elements (this is a list of length n), we see that it takes $n - 1$ key comparisons for one pass to get the largest element at the end of the list. The next pass bubbles up the second largest element. The performance is then $(n - 1) + (n - 2)$ for two passes. For n passes, the performance is the following sum

$$(n - 1) + (n - 2) + \cdots + 1. \quad \text{eq. (7-1)}$$

This is a mathematical series that sums to:

$$\frac{n(n-1)}{2} \quad \text{eq. (7-2)}$$

which is equal to:

$$\frac{n^2}{2} - \frac{n}{2} \in O(n^2). \quad \text{eq. (7-3)}$$

For a list with a large number of elements (i.e., n is large) the number of key comparisons is in the order of n^2 , because n^2 dominates. In the best case, we need just one pass to sort the list. The number of key comparisons is then in the order of n . We say that the time

complexity in the best case is $O(n)$. The time complexity in the worst-case and the average case is $O(n^2)$.

7.4. Merge sort

Merge sort is a sorting algorithm that sorts data items into ascending or descending order, which falls under the category of comparison-based sorting. The algorithm was invented by John von Neumann in 1945 (Knuth D. E., 1998) and belongs to the divide-and-conquer technique.

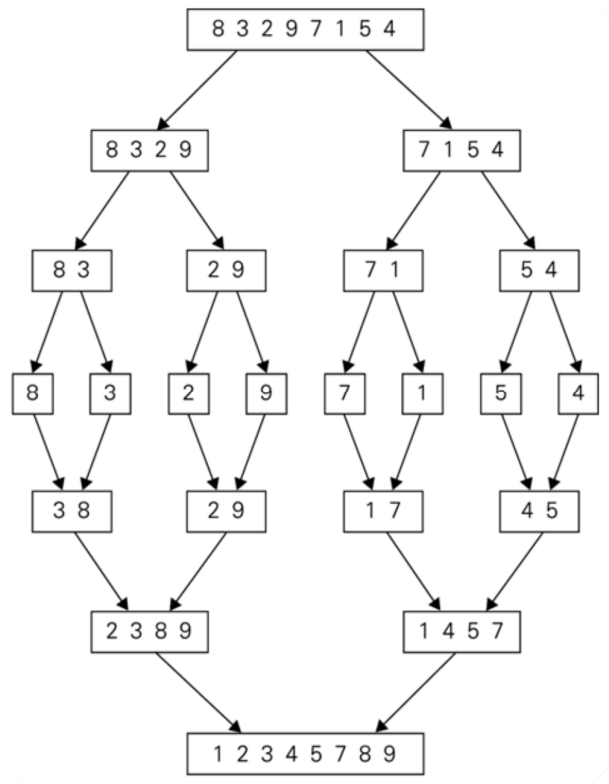


Figure 7-7: Example of merge sort (Levitin, 2012)

In the first step, merge sort divides an unsorted list into two halves or two lists if the number of elements is odd. This process repeats itself until there are N sub-lists with only one element since a list with one element is considered sorted. In the second step, each of the sub-list is sorted recursively to produce newly sorted sub-lists. In the last step, the two smaller sorted lists are merged into a single sorted list.

This last step of merging two sorted arrays or lists can be done as follows. Two pointers, i and j , are initialized to point to the first elements of the arrays to be merged. The pointed elements are compared, and the smallest is added to a new array that is being constructed. Then the index of the smaller element is incremented to point to its immediate successor in the array it was copied from. This operation is repeated until one of the two given arrays is exhausted. The remaining elements of the other array are then copied to the end of the new array, resulting in a sorted array.

7.4.1. Analysis

When we consider the performance of merge sort, we see that most implementations of this algorithm produce a stable sort and that the time complexity both in the best case and the worst case is $O(n \log(n))$. This is faster than bubble sort, insertion sort, and selection sort, which all have an $O(n^2)$. Despite this, it is worth noting that merge sort will not be faster in all cases. For example, insertion sort might be the fastest sort for all lists smaller than five elements. In practice, however, merge sort is usually faster for lists of 50 or more elements. Even so, we should bear in mind that this extra speed does not come without a price. Unlike other sort algorithms, which take a list and modify the list in place producing a sorted list, merge sort needs some additional space to merge the two last lists.

7.5. Quicksort

Quicksort (sometimes called *partition-exchange sort* (Foster, 1992)) is one of the main sorting algorithms besides merge sort and bubble sort. The algorithm was developed by Tony Hoare in 1959 (Sir Antony Hoare, 2015) and is still a widely used sorting algorithm. The quicksort algorithm is based on the divide-and-conquer technique and is one of the fastest and most efficient sorting algorithms in practical scenarios. It can be faster than merge sort and about two or three times faster than heapsort if implemented properly (Skiena S. S., 2008). Unlike merge sort, which divides the input element based on their *position* in the array, quicksort divides the elements based on their *value*. Quicksort can sort items of any type for which a "less than" relationship has been defined and is, therefore, a comparison sort algorithm. The quicksort sorting algorithm is an in-place sorting algorithm that requires small additional amounts of memory to perform the sorting.

Quicksort rearranges elements of a given array $A[0..n-1]$ to achieve its *partition*. A partition is a situation where all the elements before some position s are less than or equal to $A[s]$ and all elements after position s are greater than or equal to $A[s]$:

$$\begin{array}{ccc} & \text{Pivot} & \\ & \downarrow & \\ \underbrace{A[0] \dots A[s-1]} & A[s] & \underbrace{A[s+1] \dots A[n-1]} \\ \text{all are } \leq A[s] & & \text{all are } \geq A[s] \end{array} \quad \text{eq. (4)}$$

Obviously, after a partition has been achieved, $A[s]$ will be in its final position in the sorted array and we can continue to sort the two sub-arrays of the elements before and after $A[s]$ independently (e.g., by the same method). Because of its guiding role, we call the element $A[s]$ the *pivot*.

7.5.1. Partitioning

As we mentioned earlier, a partition is a rearrangement of the elements of the list so that all elements to the left of the pivot are less than or equal to the pivot, and all the elements to the right of the pivot are greater than or equal to the pivot. There are several alternative procedures for rearranging elements to achieve a partition. Here, we use an efficient method here based on two scans of the sub-array: a left-to-right scan and a right-to-left scan, each comparing the sub-arrays element to the pivot.

We start the partition step by selecting an element to be the pivot. Ideally, the pivot should be the median element. But it is randomly chosen to speed up the selection process. Next, we create two index pointers, for example i and j . Index pointer i starts from the beginning of the array and index pointer j starts from the end of the array. These pointers mark the positions of the elements we are scanning. Now we will scan the sub-arrays from both ends. The index pointer i does the left-to-right scan. It scans elements smaller than the pivot to be in the first part of the sub-array. This index pointer skips elements smaller than the pivot and stops upon encountering the first element greater than or equal to the pivot. The index pointer j does the right-to-left scan. It scans elements greater than the pivot to be in the second part of the sub-array. This index pointer skips elements greater than the pivot and stops when encountering the first element smaller than or equal to the pivot. After both the index pointers have stopped, we swap the elements that the pointers are marking. We continue doing this until the index pointers pass each other. Once that happens, we have divided the array into two partitions: a partition with the elements smaller than the pivot, and a partition with the elements greater than the pivot. That finishes the partition step. We repeat the partition procedure for the two separate partitions until only one element remains in the list.

7.5.2. Analysis

With quicksort, we have an efficient in-place algorithm to partition a list. The partition can be done with constant amount of extra memory, using just a few temporary variables. Note that, unlike merge sort, with quicksort we do not have to create auxiliary arrays, but we can work on the original arrays themselves. The entire work takes place in the division stage, so no additional work is required to combine the solutions for the subproblems. We just have to keep track of the start and the end index of the segments.

The time complexity of the quicksort algorithm in both the average case and best case is $O(n \log(n))$ for an array of length n . The time complexity in the worst-case is $O(n^2)$. But as we said before, quicksort is an in-place sorting algorithm, which takes up almost a constant amount of extra memory. Despite having a worst-case running time of $O(n^2)$, quicksort is quite fast and efficient in practice. The worst-case running time of quicksort is almost always avoided by using what we call a randomized version of quicksort (i.e., a quicksort version where we take a random element as a pivot). Randomized quicksort has a very high probability to give us a running time of $O(n \log(n))$.

8. Greedy technique

The *Greedy technique* is a straightforward and simple technique, which consists in making the best choice from the available alternatives at each stage, in the hope that a series of local optimal choices ultimately lead to a globally optimal solution to the main problem. This is the so-called *Greedy choice property*. A Greedy algorithm generally does not produce an optimal solution. Nevertheless, a Greedy heuristic may yield locally optimal solutions that approximate a globally optimal solution within a reasonable time.

Consider the following *change-making problem* faced by millions of cashiers all over the world: giving change for a specific amount of money n with the least number of coin denominations $d_1 > d_2 > \dots > d_m$ used at a given location (Levitin, 2012). For instance, let $d_1 = 50$, $d_2 = 20$, $d_3 = 10$, and $d_4 = 5$ be Euro cents. Using this coin denominations, how would you give change of 95 Euro cents? Most people would come up with the following answer: 1 x 50 Euro cents, 2 x 20 Euro cents, and 1 x 5 Euro cents. This strategy of making a sequence of best choices from the currently available alternatives is called *Greedy thinking*. In the first step, “Greedy” thinking leads giving one coin of 50 Euro cents, since it would reduce the remaining amount the most, namely to 45 Euro cents. For the second step, we had the same coin denominations at our disposal, however we could not give 50 Euro cents as that would violate the problem’s constraints. That being the case, the best selection in this step was two 20 Euro cents. This step reduced the remaining amount to 5 Euro cents. Finally, giving 5 Euro cents, can further reduce the owned amount to zero, solving the problem. The question now is: is this the optimal solution to this problem? And the answer is “Yes, it is.”

To use the Greedy technique the algorithm should meet the following requirements for each step,

- Feasible: i.e., the choice made must meet the problem’s constraints.
- Local optimal: i.e., it must be the best local choice out of all feasible choices available for that step.
- Irrevocable: i.e., once the choice has been made, it cannot be changed in subsequent steps of the algorithm. In other words, a greedy algorithm never reconsiders its choices, no matter what situation arises later.

Despite its simplicity, the Greedy technique has one considerable drawback. Namely, it usually (but not always) fails to find the globally optimal solution, since it is not exhaustive on all data. Greedy algorithms are best suited for simple problems (e.g., the change-making problem).

If a Greedy algorithm can be proven to yield the global optimum for a given problem class, it typically becomes the method of choice. This is because it is faster than other optimization methods such as dynamic programming. Examples of such algorithms are Dijkstra’s

algorithm for finding single-source shortest path and Prim's algorithm for finding minimum spanning trees.

Greedy algorithms are often used in ad hoc mobile networking to efficiently route packets with the fewest number of hops and the shortest possible delay. They are also used in business intelligence (BI), artificial intelligence (AI), machine learning, and programming.

9. Introduction to graph theory

9.1. Introduction

In computer science, graphs are used to represent networks of communication, data organization, computational devices, the flow of computation, etc. A practical example would be using a directed graph to represent the link structure of a website. The vertices would be the web pages available on the website. A directed edge from page A to page B would exist if and only if A contains a link to B (Bhulai, 2011). A similar approach could be taken to problems in many other fields such as travel, biology, or computer chip design. Therefore, the development of algorithms to handle graphs is of great importance in computer science.

9.2. Graphs

Informally, a graph can be thought of as a set of *vertices* or *nodes* and a set of *edges* or *arcs* which connect pairs of vertices (Levitin, 2012). See Figure 9-1 for an example of a graph.

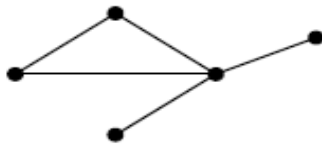


Figure 9-1: An undirected graph

Formally, a graph $G = (V, E)$ is defined by a pair of two sets:

- 1) a finite set V of items called *vertices*, and
- 2) a set E of pairs of these items called *edges*.

Now suppose we have two vertices u and v , i.e., $V = \{u, v\}$. Then the pair of vertices (u, v) is the same as the pair (v, u) if these pairs of vertices are unordered. We say that the vertices u and v are *adjacent* to each other and that they are connected by the *undirected edge* (u, v) . We call the vertices u and v *endpoints* of the edge (u, v) . We write $e = (u, v)$ if edge $e \in E$ joins $u, v \in V$. A graph G is called *undirected* if every edge $e \in E$ is undirected. This means that every edge in graph G is undirected. An undirected graph is *symmetric*. Figure 9-1 is an example of an undirected graph.

We say that the edge $e = (u, v)$ is *directed* from the vertex u to vertex v if a pair of vertices (u, v) is not the same as the pair (v, u) . Vertex u is called the edge's *tail* and vertex v is called the edge's *head*. A graph G whose every edge e is directed is called a *directed graph*. Figure 9-2 shows an example of a directed graph G . Directed graphs are also called *digraphs* and are *asymmetric*.

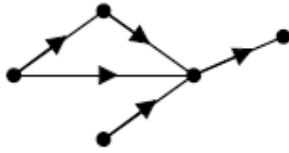


Figure 9-2: A directed graph

A graph G with both directed and undirected edges is called a *mixed graph*. Figure 9-3 is an example of a mixed graph.

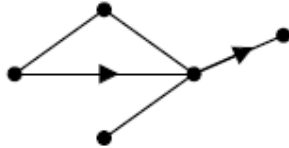


Figure 9-3: A mixed graph

Vertices of a graph are normally labeled with letters, integer numbers, or if an application calls for it, character strings. The graph shown in Figure 9-4 has six vertices and seven undirected edges:

$$V = \{a, b, c, d, e, f\},$$

$$E = \{(a, b), (a, d), (b, c), (b, e), (c, f), (e, d), (e, f)\}.$$

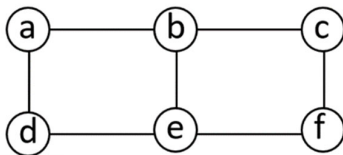


Figure 9-4: An undirected graph with labeled vertices

9.3. Representations of graphs

Graphs in computer science are usually represented by the so-called *adjacency matrices*. The adjacency matrix of a graph with n vertices is a $n \times n$ Boolean matrix with one row and one column for each of the graph's vertices. In this matrix, the element in the i – th row and the j – th column is equal to 1 if there is an edge from the i – th vertex to the j – th vertex, and equal to 0 if there is no such edge. For example, the graph of Figure 9-4 is represented by the adjacency matrix in Figure 9-5.

$$\begin{array}{c}
 \begin{array}{ccccc}
 & a & b & c & d & e & f \\
 a & \left(\begin{array}{cccccc}
 0 & 1 & 0 & 1 & 0 & 0 \\
 1 & 0 & 1 & 0 & 1 & 0 \\
 0 & 1 & 0 & 0 & 0 & 1 \\
 1 & 0 & 0 & 0 & 1 & 0 \\
 0 & 1 & 0 & 1 & 0 & 1 \\
 0 & 0 & 1 & 0 & 1 & 0
 \end{array} \right)
 \end{array}
 \end{array}$$

Figure 9-5: Adjacency matrix

Note that the adjacency matrix of an undirected graph is always symmetric. This means that $A[i, j] = A[j, i]$, where $A[i, j]$ is one way to represent a matrix mathematically.

9.4. Some examples of graphs

In this section, we will briefly discuss some examples of graphs.

9.4.1. Weighted graphs

We call a graph a *weighted graph* if the graph has numbers assigned to its edges. In Figure 9-6 we see an example of a weighted graph.

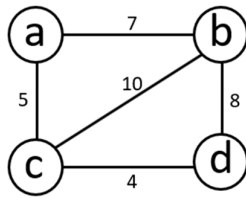


Figure 9-6: Weighted graph

The numbers assigned to the edges are called *weights* or *costs*. Weighted graphs are crucial for many real-life applications, such as finding the shortest path between two points in a transport or communication network. Dijkstra's and Prim's algorithms use weighted graphs. We will discuss these algorithms in Section 10. The matrix of a weighted graph is called a *weighted matrix* or a *cost matrix* (see Figure 9-7 for an example).

$$\begin{array}{c}
 \begin{array}{cccc}
 & a & b & c & d \\
 a & \left(\begin{array}{cccc}
 \infty & 7 & 5 & \infty \\
 7 & \infty & 10 & 8 \\
 5 & 10 & \infty & 4 \\
 \infty & 8 & 4 & \infty
 \end{array} \right)
 \end{array}
 \end{array}$$

Figure 9-7: Weighted matrix

A weighted matrix contains the weight of the edge from the i -th to the j -th vertex if there is such an edge and an infinity symbol (∞) if there is no such edge. So, for example, in Figure 9-6 the edge from vertex a to vertex b has a weight of 7 in the graph, then we write 7 in the matrix from vertex a to vertex b (see Figure 9-7). There is no edge from vertex a to vertex d in the graph, so we write an infinity symbol, ∞ , in the matrix from vertex a to vertex d . We say that Figure 9-7 is the weighted matrix of the weighted graph of Figure 9-6.

9.4.2. Simple graphs

A graph is called a *simple* or *strict graph* (Tutte, 1998) (see Figure 9-8) if it is an undirected graph with no weights, no loops, and no more than one edge between any two different vertices. A simple graph may be either connected or disconnected.

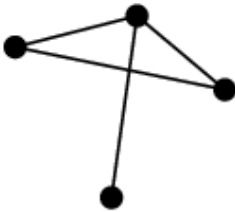


Figure 9-8: Simple graph

An edge of a graph which joins a vertex to itself is called a *loop*.

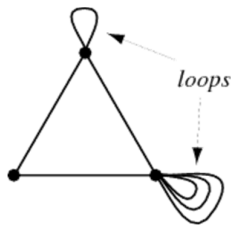


Figure 9-9: Graph with loops (Weisstein, 2015)

A simple graph cannot contain any loops. Unless otherwise stated, the term “graph” without descriptors usually refers to a simple graph.

9.4.3. Multigraph

Multiple edges are two or more edges connecting the same two vertices within a simple graph. Such a simple graph with multiple edges is called a *multigraph* (see Figure 9-10).

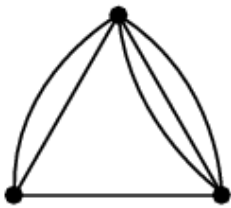


Figure 9-10: Multigraph

9.4.4. Connected and disconnected graphs

A graph is called *connected* if there is, for any two given vertices u and v , a path from u to v . Otherwise, it is called *disconnected*. The graph in Figure 9-12 is disconnected as there is no path from, for example, vertex e to vertex a or from vertex f to vertex c .

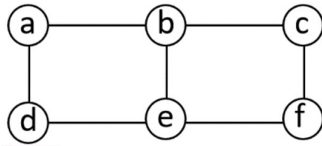


Figure 9-11: Undirected connected graph

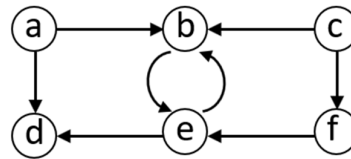


Figure 9-12: Directed disconnected graph

A *path* in the graph theory is defined as a finite or infinite sequence of edges that connect a sequence of vertices that are all distinct from each other (Bender & Williamson, 2010).

9.4.5. Connected component

A *connected component* of an undirected graph is a subgraph in which any two vertices are connected by paths, and which is connected to no additional vertices in the supergraph (Connected component, n.d.). The graph in Figure 9-13 consists of three components and is called the supergraph. Note that the components are also graphs which are called the subgraphs. For example, the path from u to v is shown in red.

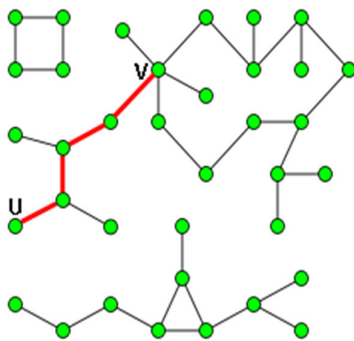


Figure 9-13: Graph with three connected components

9.4.6. Complete graphs

A *complete graph* on n vertices (denoted as K_n) is a graph in which all vertices are mutually connected by means of direct edges. A complete graph is simple and undirected. To make a graph with n vertices complete you need $n - 1$ edges on each vertex. For example, to make a graph with 3 vertices complete, you need 2 edges from each vertex. Likewise, you need 3 edges from each vertex to make a graph with four vertices complete (see Figure 9-14 for examples). We say that K_n is a graph of *degree* $n - 1$.

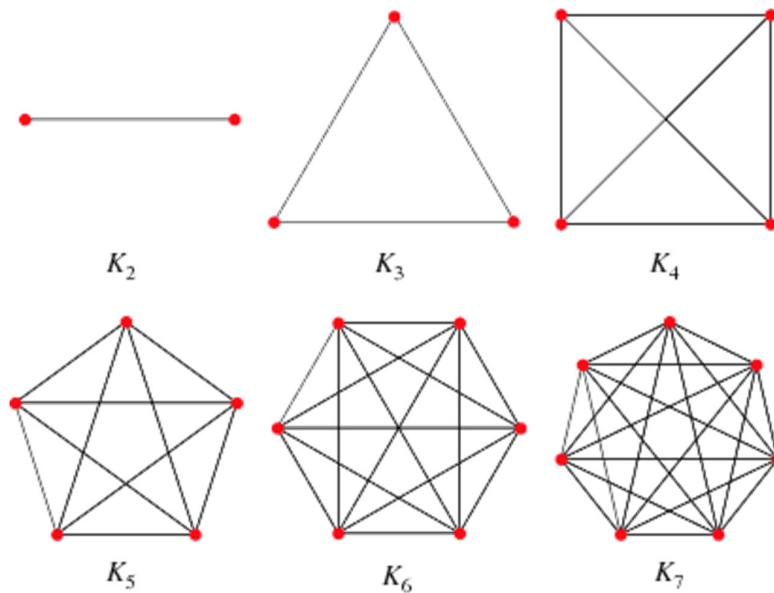


Figure 9-14: Complete graphs

A complete graph with n vertices has $\frac{n(n-1)}{2}$ edges.

9.4.7. Cycle graphs

A *cycle graph* or a *circular graph* on n vertices (denoted as C_n) is a simple connected graph with n vertices where each vertex is connected with two other vertices. Note that the cycle graph has a close path that starts and ends at the same vertex and does not traverse the same edge more than once. Figure 9-15 illustrates a cycle graph in which the vertices a, b, c, d, a form a cycle. A graph with no cycles is called *acyclic*.

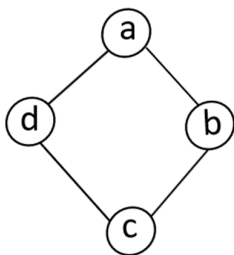


Figure 9-15: Cycle graph

The number of vertices in C_n equals the number of edges. Every vertex has degree 2, which means that every vertex has exactly two edges.

9.4.8. Trees

A *tree* is a simple, undirected, connected, acyclic graph (see Figure 9-16). In a tree, any two vertices are connected by exactly one path. A tree with n vertices has $n - 1$ edges. A graph that has no cycles but is not necessarily connected is called a *forest* (see Figure 9-17). This means that each of its connected components is a tree.

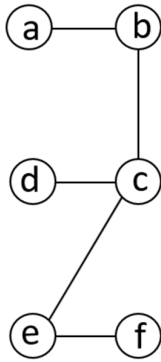


Figure 9-16: Tree

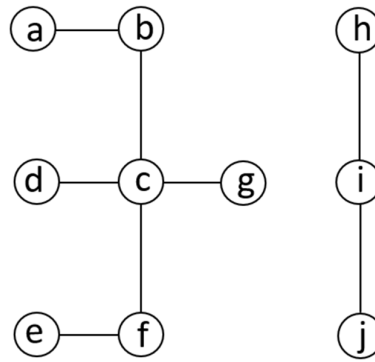


Figure 9-17: Forest

9.4.9. Eulerian graph

We say that a path is an *Eulerian path* or *Eulerian trail* if we can find a path in a connected graph by visiting every edge exactly one time. In other words, an Eulerian path is a path in a connected graph that contains all the edges of the graph. An *Eulerian cycle* or an *Eulerian circuit* is an Eulerian path that starts and ends at the same vertex, which means that we have a circuit in a graph that contains all of the edges of the graph. We call a graph an *Eulerian graph* if it has an Eulerian cycle. The pentatope graph in Figure 9-18 is an example of an Eulerian graph because it contains an Eulerian cycle since we can find a circuit in this graph that contains all the edges of the graph.

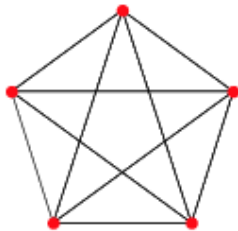


Figure 9-18: Pentatope graph

The Eulerian graph is a special kind of graph that was identified by the Swiss mathematician and physicist Leonhard Euler (1707 - 1783) (Leonhard Euler, n.d.) following his solution to the problem of the famous Seven Bridges of Königsberg in 1736. Mathematically the problem can be stated as follows:

Given the graph of the Königsberg Bridges (Figure 9-19), is it possible to construct a path (or a cycle) that visits each edge exactly once?

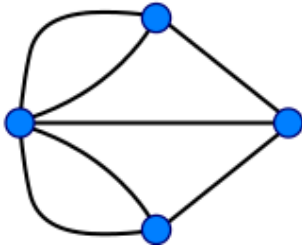


Figure 9-19: Graph of the Königsberg Bridges

Euler proved that in order for a graph to contain an Eulerian circuit, all vertices must have an even degree. He stated without proof that connected graphs with all vertices of even degree have an Eulerian circuit.

9.4.10. Hamiltonian graph

A *Hamiltonian path*, also called a *Hamilton path* or *traceable path*, is defined as a path in an undirected or directed graph that visits each vertex exactly once. A *Hamiltonian cycle* - also called *Hamiltonian circuit*, *Hamilton cycle*, or *Hamilton circuit* - is a graph cycle (i.e., closed loop) through a graph that visits each vertex exactly once (Skiena & Pemmaraju, 2003). In other words, a Hamiltonian cycle is a path that visits all the vertices of the graph exactly once before returning to the starting vertex for this graph. We call a graph a *Hamiltonian graph* or a *Hamilton graph* if it has a Hamiltonian cycle.

Hamiltonian paths and cycles are named after the Irish physicist, astronomer, and mathematician Sir William Rowan Hamilton (1805 - 1865) (Hankins, 1980) who invented the *Icosian Game* in 1857 (Darling, 2016). This mathematical game was played on a circular wooden game board on which a graph (See Figure 9-20) was carved.

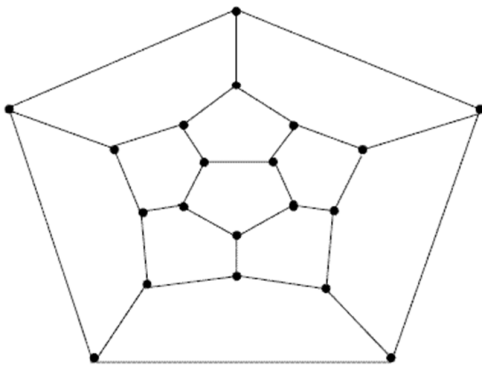


Figure 9-20: Graph for the Icosian Game

The aim of the game was to find a Hamiltonian cycle along the edges of a dodecahedron (See Figure 9-20) so that every vertex was visited once and the endpoint is the same as the starting point.

10. Graph algorithms

10.1. Dijkstra's algorithm

There are several well-known algorithms to find the shortest paths. In this section, we consider the best-known algorithm for the single-source shortest path problem, Dijkstra's algorithm. The single-source shortest path problem consists in finding the shortest path (i.e., the path with the lowest cost) from one designated node (vertex), called the source, to every other node in a weighted connected graph (e.g., a road network) such that the sum of the weights of its constituent edges is minimized. Dijkstra's algorithm is a graph search algorithm that solves the single-source shortest path problem and belongs to the Greedy techniques. It works in both directed and undirected graphs. However, it only works in graphs with non-negative edge weights (e.g., distance or time). The algorithm was developed in 1956 by the Dutch computer scientist Edsger W. Dijkstra who lived from 1930 to 2002 (Edsger W. Dijkstra, 2015).

Dijkstra's algorithm comes in many variants. The original variant found the shortest path between two nodes (Dijkstra, 1959), but a more common variant fixes a single node as the "source" node and finds the shortest paths from the source to all other nodes in the graph, producing a *shortest path tree*⁵. The algorithm can also be used to find the shortest paths from a single node to a single destination node by stopping the algorithm once the shortest path to the destination node has been determined. For example, if the nodes of the graph represent cities and edge path costs represent driving distances between pairs of cities connected by a direct road, Dijkstra's algorithm can be used to find the shortest route between a city and all other cities. As a result, the shortest path algorithm is widely used in packet routing in communication networks (the Internet included), transportation planning and as a subroutine in other graph algorithms (Dijkstra's algorithm, 2015).

10.1.1. How does Dijkstra's algorithm work?

To understand how Dijkstra's algorithm works we consider the example in Figure 10-1.

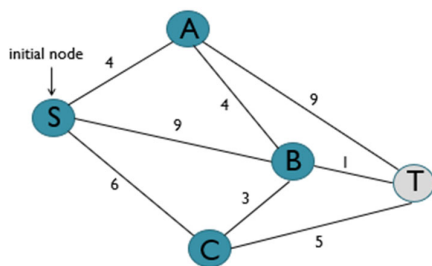


Figure 10-1: The initial node in the application of Dijkstra's algorithm.

⁵ Given a connected, undirected graph G , a shortest path tree rooted at node v is a spanning tree T of G , such that the path distance from root v to any other node u in T is the shortest path distance from v to u in G (Cahn, 1998).

In this graph, the nodes may, for example, represent cities or computers, and the edges will represent then the distance between the cities or the connections between the computers. Now, we want to find the shortest path from source S to destination T . Let's call the node we are starting at the *initial node* in the graph. In the example we are starting at node S , so node S is the initial node. And let's call the distance from the initial node to a node X , the distance of node X . So, the distance of node A , in the example, is 4. This is because the distance starting from node S , the initial node, to node A is 4. Dijkstra's algorithm will assign some initial distance values and will try to improve these step by step which we list below (Dijkstra's algorithm, 2015).

1. The first step in Dijkstra's algorithm is to assign every node a tentative distance value. The distance value is set to zero for the initial node since the distance to the source is zero, and the distance is set to infinite for all the other nodes.
2. In the second step, a set of all the unvisited nodes is created, called the *unvisited set* and the initial node is set as the current node.
3. The next step is to consider all of the unvisited neighbors for the current node and calculate their *tentative* distance for the current node, which is the sum of the weighted edges on the path from the source. The newly calculated *tentative* distance to the currently assigned value is compared and the smaller one is assigned. For example, if the current node A is marked with a distance of 4, and the edge connecting it to a neighbor B has a length of 4, then the distance to B (via A) will be $4 + 4 = 8$. Update the distance to 8 if B was previously marked with a distance greater than 8. Otherwise, keep the current value.
- 4.
5. When we are done considering all of the neighbors of the current node, mark the current node as visited and remove it from the *unvisited set*. A visited node will never be checked again.
6. If the destination node has been marked visited (when planning a route between two specific nodes) or if the smallest tentative distance among the nodes in the *unvisited set* is infinity (when planning a complete traversal; occurs when there is no connection between the initial node and remaining unvisited nodes), then stop. The algorithm has finished.
7. Otherwise, select the unvisited node that is marked with the smallest tentative distance, set it as the new "current node", and go back to step 3.

10.2. Prim's algorithm

We need to define some terms before considering Prim's algorithm in this section.

DEFINITIONS

A **spanning tree** of a connected graph is its connected acyclic sub-graph (i.e., a tree) that contains all the vertices of the graph.

A **minimum spanning tree** of a weighted connected graph is its spanning tree of the smallest weight, where the **weight** of a tree is defined as the sum of the weights on all its edges.

The **minimum spanning tree problem** is the problem of finding a minimum spanning tree for a given weighted connected graph.

Suppose that we have the following practical problem: connect n points so that there is a path between every pair of points in the cheapest possible way. In this problem, the points can be represented by vertices of a graph, the possible connections can be represented by the edges of the graph, and the connection costs can be represented by the edge weights. Then the problem can be posed as the minimum spanning tree problem.

Prim's algorithm finds a *minimum spanning tree* for a connected weighted undirected graph and belongs to the Greedy techniques. It finds a subset of the edges forming a tree that includes every vertex, and where the total weight of all the tree edges is minimized. The algorithm was first discovered in 1930 by Czech mathematician Vojtěch Jarník and later independently rediscovered and republished by the computer scientists Robert Clay Prim in 1957 and Edsger W. Dijkstra in 1959. The algorithm is sometimes referred to as the *DJP algorithm* or the *Jarník algorithm* (Robert C. Prim, n.d.).

Prim's algorithm works by constructing the minimum spanning tree through a sequence of expanding sub-trees. The initial sub-tree in such a sequence consists of a single vertex, which is selected randomly from the set V of vertices. In each iteration, we expand the current tree by adding the cheapest possible connection from the tree to another vertex.

The algorithm can be informally described as performing the following steps:

1. Select any vertex to be the first of the tree T .
2. Consider which edge connects vertices in T to vertices outside T . Pick the edge with the minimum weight (if there are more than one, then choose any). Add this edge and vertex to T .
3. Repeat step 2 until T contains every vertex of the graph.

For any connected weighted undirected graph, Prim's algorithm will always yield a minimum spanning tree (Prim's_algorithm: Proof_of_correctness, n.d.).

10.3. Kruskal's algorithm

In the previous section, we considered Prim's algorithm that "grows" a minimum spanning tree through a greedy inclusion of the nearest vertex to the vertices already in the tree. In this section, we will consider Kruskal's algorithm.

Kruskal's algorithm is named after the American mathematician, statistician, computer scientist and psychometrician Joseph Bernard Kruskal (1928 - 2010). He discovered the algorithm when he was a second-year graduate student (Kruskal, 1956). Kruskal's algorithm, like Prim's algorithm, is also a greedy algorithm applied to graph theory to find a minimum spanning tree for a connected weighted graph. If the graph is not connected, it finds a minimum spanning forest (i.e., a minimum spanning tree for each connected component). The algorithm looks at a minimum spanning tree of a connected weighted graph $G = (V, E)$ as an acyclic subgraph with $|V| - 1$ edges for which the sum of the edge weights is the smallest. Consequently, the algorithm constructs a minimum spanning tree as an expanding sequence of subgraphs that are always acyclic but not necessarily connected on the intermediate stages of the algorithm (Levitin, 2012).

The algorithm starts by sorting the graph's edges in non-decreasing weight order, creating a sorted list of edges. Then, starting with the empty subgraph, it scans this sorted list, adding the next edge of the list to the current subgraph if such inclusion does not create a cycle and simply skipping the edge otherwise.

The algorithm can be informally described as performing the following steps:

1. Remove all loops.
2. Remove all parallel edges between two vertices except the one with the least weight.
3. Create an edge table. An edge table consists of all the edges along with their weight in ascending order, i.e. a sorted list of edges.
4. Select the edge with the minimum weight.
5. Select the next edge with the minimum weight that does not form any cycle with previously selected edges. You may have a forest now.
6. Repeat step 5 until T contains every vertex of the graph.

10.3.1. Comparison of Prim's and Kruskal's algorithm

At first sight, both Prim's and Kruskal's algorithm seems to operate in the same way. Both Prim's algorithm and Kruskal's algorithm are greedy algorithms for finding the minimum spanning tree. However, when we compare both algorithms, we see some differences between them.

- a) For Prim's algorithm, the graph *must* be connected, but that is not true in the case of Kruskal's algorithm.

- b) Prim's algorithm only grows one tree, while Kruskal's algorithm grows a collection of trees (a forest).
- c) In Prim's algorithm, the next edge in the minimum spanning tree shall be the cheapest edge in the *current vertex*. In Kruskal's algorithm, we shall choose the cheapest edge, but it may not be in the current vertex.
- d) Prim's algorithm appears to run faster in dense graphs with more edges than vertices, while Kruskal's algorithm appears to run faster in sparse graphs.
- e) The running time of Kruskal's algorithm is $O(E \log E)$, or equivalently, $(E \log V)$, where E is the number of edges and V is the number of vertices. This can be reached by simple data structures⁶. The time complexity of Prim's algorithm depends on the data structures used for the graph and for ordering the edges by weight. A simple implementation of Prim's algorithm, using an adjacency matrix, requires $O(|V|^2)$ running time.

⁶ A data structure is a particular way of organizing data in a computer so that it can be used efficiently. Different kinds of data structures are suitable for different kinds of applications. Data structures provide a means to efficiently manage large amounts of data for applications such as large databases and Internet indexing services. Usually, efficient data structures are key to designing efficient algorithms. Examples of data structures are *arrays or lists, graphs, trees, classes, and records* (Data structure, n.d.).

References

- Adam, A. (n.d.). *Study Habits*. Retrieved 03 11, 2015, from Study Habits: <http://www.study-habits.com/how-to-prepare-for-an-exam>
- BBC. (n.d.). *Introduction to computational thinking*. Retrieved December 2020, from <https://www.bbc.co.uk/bitesize/guides/zp92mp3/revision/1>
- Bender, E. A., & Williamson, S. G. (2010). *Lists, Decisions and Graphs. With an Introduction to Probability*.
- Bhulai, A. (2011). *Dynamic website optimization through autonomous management of design patterns*. Amsterdam: Universal Press.
- Big O notation*. (n.d.). Retrieved 03 16, 2016, from Wikipedia - The Free Encyclopedia: https://en.wikipedia.org/wiki/Big_O_notation
- Cahn, R. S. (1998). *Wide Area Network Design: Concepts and Tools for Optimization*. (1st ed.). Morgan Kaufmann.
- Centre, W. G. (n.d.). *Farm Shop*. Retrieved 03 11, 2015, from Woodcote Green Garden Centre: <http://www.woodcotegreen.com/Garden-Centre/farm-shop.aspx>
- Cherry, K. (n.d.). *What is a heuristic?* (About) Retrieved 05 19, 2015, from <http://psychology.about.com/od/hindex/g/heuristic.htm>
- Connected component*. (n.d.). Retrieved 06 08, 2015, from http://en.wikipedia.org/wiki/Connected_component_%28graph_theory%29
- Cormen, T. H., Leiserson, C. E., & Rivest, R. L. (2009). *Introduction to algorithms* (3rd ed.). Cambridge/MA: MIT Press. Retrieved from https://archive.org/details/introductiontoal00corm_805
- Darling, D. (2016, April 23). *Icosian Game*. Retrieved from David Darling: https://www.daviddarling.info/encyclopedia/I/Icosian_Game.html
- Data structure*. (n.d.). Retrieved 05 25, 2015, from https://en.wikipedia.org/wiki/Data_structure
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1, 269–271.
- Dijkstra's algorithm*. (2015). Retrieved 05 24, 2015, from http://en.wikipedia.org/wiki/Dijkstra%27s_algorithm
- Edsger W. Dijkstra*. (2015). Retrieved 05 19, 2015, from http://en.wikipedia.org/wiki/Edsger_W._Dijkstra
- Foster, C. (1992). *Algorithms, Abstraction and Implementation*.
- Graham, R. L., Knuth, D. E., & Patashnik, O. (1994). *Concrete Mathematics*. (2nd, Ed.) Massachusetts: Addison-Wesley.
- Graph loop*. (n.d.). (Wolfram Mathworld) Retrieved 06 07, 2015, from <http://mathworld.wolfram.com>
- Hankins, T. L. (1980). *Sir William Rowan Hamilton*. Baltimore: John Hopkins University Press.
- Heuristic*. (n.d.). (vocabulary.com) Retrieved 05 19, 2015, from <http://www.vocabulary.com/dictionary/heuristic>

Icosian game. (n.d.). Retrieved 06 12, 2015, from https://en.wikipedia.org/wiki/Icosian_game

Kiriluk-Hill, R. (n.d.). *Fairness Of Hunterdon Central High School's Courtesy Busing Questioned*. Retrieved 03 11, 2015, from True Jersey (nj.com): http://www.nj.com/hunterdon/index.ssf/2009/03/fairness_of_hunterdon_central.html

Knuth, D. E. (1996). In *Selected Papers on Computer Science*. Stanford, California: Cambridge University Press.

Knuth, D. E. (1998). *The Art of Computer Programming* (2nd ed.). Addison-Wesley.

Kruskal, J. B. (1956). On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7, 48-50.

Landau, E. (1909). In *Handbuch der Lehre von der Verteilung der Primzahlen* (p. 883). Leipzig: B. G. Teubner.

Leonhard Euler. (n.d.). Retrieved 06 12, 2015, from https://en.wikipedia.org/wiki/Leonhard_Euler

Levitin, A. (2012). London, England: Pearson Education Limited.

Lundqvist, I. K. (n.d.). *Massachusetts Institute of Technology*. Retrieved 05 12, 2015, from <http://web.mit.edu/>

Matrix chain multiplication. (n.d.). Retrieved 10 07, 2021, from Wikipedia: https://en.wikipedia.org/wiki/Matrix_chain_multiplication

Matrix Chain Multiplication | DP-8. (2021, 08 07). Retrieved 10 07, 2021, from GeeksforGeeks: <https://www.geeksforgeeks.org/matrix-chain-multiplication-dp-8/>

Merge Sort. (n.d.). Retrieved 08 29, 2017, from https://en.wikipedia.org/wiki/Merge_sort

Mohr, A. (2010). *Quantum Computing in Complexity Theory and Theory of Computation*. Retrieved 03 2016, from http://www.austinmohr.com/Work_files/complexity.pdf

Path. (n.d.). Retrieved 06 08, 2015, from http://en.wikipedia.org/wiki/Path_%28graph_theory%29

Prim's_algorithm: Proof_of_correctness. (n.d.). Retrieved 05 30, 2015, from http://en.wikipedia.org/wiki/Prim%27s_algorithm#Proof_of_correctness

Robert C. Prim. (n.d.). Retrieved 05 29, 2015, from http://en.wikipedia.org/wiki/Robert_C._Prim

Rowell, E. (n.d.). *Know Thy Complexities!* Retrieved 05 16, 2015, from <http://bigocheatsheet.com/>

Search algorithm. (n.d.). Retrieved 04 24, 2015, from Wikipedia: http://en.wikipedia.org/wiki/Search_algorithm

Sir Antony Hoare. (2015, April 15). Retrieved from Computer History Museum: <https://web.archive.org/web/20150403184558/http://www.computerhistory.org/feilowawards/hall/bios/Antony%2CHoare/>

Skiena, S. S. (2008). *The Algorithm Design Manual*. Springer.

- Skiena, S., & Pemmaraju, S. (2003). *Computational Discrete Mathematics: Combinatorics and Graph Theory with Mathematica*. United States of America: Cambridge University Press.
- Time complexity*. (n.d.). Retrieved 05 05, 2015, from http://en.wikipedia.org/wiki/Time_complexity
- Tony Hoare*. (n.d.). Retrieved 08 29, 2017, from https://en.wikipedia.org/wiki/Tony_Hoare
- Tutte, W. T. (1998). *Graph Theory as I Have Known It*. Oxford, England: Oxford University Press.
- Weisstein, E. W. (2015, May 15). *Graph Loop*. Retrieved from MathWorld - A Wolfram Web Resource: <https://mathworld.wolfram.com/GraphLoop.html>
- William Rowan Hamilton*. (n.d.). Retrieved 06 12, 2015, from https://en.wikipedia.org/wiki/William_Rowan_Hamilton
- Wing, J. (2014). Computational Thinking Benefits Society. *40th Anniversary Blog of Social Issues in Computing*.

Index

A

<i>acyclic</i>	40
adjacency matrices	36
adjacent	35
algorithm	13
arc	35
asymmetric graphs	35

B

back reasoning	10
basic operations	19
big O	17
binary search algorithm	24, 25
brute force solution strategy	10, 11
bubble sort	11, 18, 19, 28, 30

C

change-making problem	<i>See</i> Greedy technique
circular graph	<i>See</i> cycle graph
complete graph	39
connected component	39
connected graph	38
constant time	19
cost matrix	<i>See</i> weighted matrix
costs	<i>See</i> weights
cycle graph	40

D

data structures	47
<i>degree</i>	39
digraphs	35
Dijkstra's algorithm	43
directed	35
directed graph	35
disconnected graph	38
discover a structure or pattern solution strategy	10
divide the problem into several sub problems solution strategy	10
divide-and-conquer solution strategy	10
divide-and-conquer solution strategy (D&C)	11
divide-and-conquer technique	29, 30
DJP algorithm	45

E

edge	35
endpoints	35
Eulerian circuit	<i>See</i> Eulerian cycle

Eulerian cycle	41
Eulerian graph	41
Eulerian path	41
<i>Eulerian trail</i>	<i>See</i> Eulerian path
exclusion	10

F

forest	40
--------------	----

G

go through all the possibilities solution strategy	10
graph	35
Greedy algorithm	33
Greedy choice property	33
Greedy technique	33, 43, 45
Greedy thinking	33
guess and check solution strategy	10

H

half-interval search algorithm	<i>See</i> Binary search algorithm
Hamilton circuit	<i>See</i> Hamiltonian cycle
Hamilton cycle	<i>See</i> Hamiltonian cycle
Hamilton graph	<i>See</i> Hamiltonian graph
Hamilton path	<i>See</i> Hamiltonian path
Hamiltonian circuit	<i>See</i> Hamiltonian cycle
Hamiltonian cycle	42
Hamiltonian graph	42
head	35
heuristic	11

I

Icosian Game	42
in place algorithm	28
<i>initial node</i>	44
input size	<i>See</i> problem size

J

Jarník algorithm	45
------------------------	----

K

<i>key</i>	23, 24, 25, 28
Kruskal's algorithm	46

L

Landau's symbol	<i>See</i> Big O
Linear search	23
loop	16

M

make a model solution strategy	10, 11
merge sort	12, 29, 30, 31
minimum spanning tree	45
minimum spanning tree problem	45
mixed graph	36
multigraph	38
Multiple edges	38

N

node	35
------------	----

P

Partition	30
partition-exchange sort	<i>See</i> quicksort
path	39
pentatope graph	41
permutation	27
pivot	30, 31
Prim's algorithm	34, 45, 46
problem size	19
pseudocode	15

Q

quicksort	30, 31
-----------------	--------

R

Randomized quicksort	31
ripple sort	<i>See</i> bubble sort

S

sequential search	<i>See</i> Linear search
Seven Bridges of Königsberg	41
shortest path tree	43
simple graph	38
simplifying	10

single-source shortest path problem	43
---	----

Solution strategies

easy solution	8
efficient solution	8
systematically	8
solution strategy	8
spanning tree	45
strict graph	<i>See</i> simple graph
subexponential	18
sub-graphs	39
super-graph	39
superpolynomial	18
symmetric graphs	35

T

tail	35
time complexity	18, 19, 21, 22, 25, 29, 30, 31
traceable path	<i>See</i> Hamiltonian path
<i>tree</i>	40
try something solution strategy	<i>See</i> Guess and check solution strategy

U

undirected edge	35
undirected graph	35
unvisited set	44
use formulas or equations solution strategy	10

V

vertex	35
--------------	----

W

weight	37, 45
weighted graph	37
weighted matrix	37
worst-case time	19